

WordPress React Theme Compiler — Product Roadmap & Work Plan

Project Name (Working Titles)

Possible names:

- WPStack
 - ForgeWP
 - ReactWP
 - ThemeForge
 - WP Modern
 - StackPress
 - NovaWP
 - TailPress Studio
 - WP Fusion
 - ArtisanWP
-

1. Product Introduction

What This Product Is

This project is a modern frontend framework and theme compiler for WordPress.

It allows developers to build WordPress themes using modern JavaScript tooling such as:

- React
- TypeScript
- Tailwind CSS
- shadcn/ui components
- Vite

and then compile/export the project into a native installable WordPress theme.

The goal is to modernize WordPress frontend development without requiring developers to deeply learn PHP-based theme architecture.

2. The Problem

Modern frontend developers avoid WordPress because:

- WordPress theme development is heavily PHP-based
- Existing builders are restrictive
- Developer experience is outdated
- Styling systems are inconsistent
- Reusability is poor
- Frontend tooling feels disconnected from modern standards

Current WordPress builders focus mostly on:

- drag-and-drop editing
- no-code users
- visual design

But there is a major gap:

developer-first WordPress frontend engineering.

3. Vision Statement

Core Vision

Build WordPress themes like modern web applications.

This framework should provide:

- modern frontend tooling
- reusable component architecture
- fast development workflow
- Tailwind-native styling
- installable WordPress output
- transparent editable code
- excellent developer experience

without replacing WordPress itself.

4. Product Philosophy

The product philosophy is inspired by:

- Laravel
- shadcn/ui
- Vite

Core principles:

- convention over configuration
 - developer-first
 - editable source code
 - simple architecture
 - composable components
 - transparent output
 - performance-first
 - native WordPress compatibility
-

5. What Makes This Product Unique

Existing Tools Focus On:

- visual editing
- no-code builders
- proprietary systems
- plugin lock-in

This Product Focuses On:

- developers
- React workflows
- Tailwind styling
- reusable UI systems
- source-controlled themes
- modern frontend architecture

6. Core Features

Phase 1 Features

- React-based development
- Vite-powered dev server
- Tailwind CSS support
- shadcn-style component installer
- theme compilation
- WordPress theme ZIP export
- hot reload during development

Future Features

- Gutenberg block support
- WooCommerce support
- WP data hooks
- dynamic content binding
- component marketplace
- visual editor
- AI-assisted scaffolding
- theme deployment pipeline

7. Core Product Architecture

Development Layer

Developer builds using:

- React
- TypeScript
- Tailwind
- reusable components

Compilation Layer

Compiler transforms:

- React pages
- layouts
- assets
- metadata

into:

- PHP templates
 - WordPress theme structure
 - installable assets
-

Output Layer

Generated output:

- installable WordPress theme
 - optimized assets
 - theme.json
 - templates
 - compiled CSS/JS
-

8. Technical Direction

Recommended Stack

Core

- TypeScript
- Node.js
- Vite
- React

Styling

- Tailwind CSS
- PostCSS

Compiler

- Babel / SWC
- AST transformation
- file generation

Packaging

- ZIP generation
 - manifest builder
-

9. Architecture Principles

Important Principle #1

This product does NOT run Next.js inside WordPress.

Instead:

React is used as the authoring layer.

The framework compiles React projects into native WordPress-compatible themes.

Important Principle #2

The framework should minimize runtime JavaScript where possible.

Goals:

- fast loading
- low hydration
- SEO-friendly output
- minimal frontend overhead

Important Principle #3

Avoid proprietary lock-in.

Generated code should remain:

- editable
- portable
- transparent

10. Recommended MVP Scope

MVP Goal

Create the smallest possible usable version.

MVP Features

- CLI project creation
- React + Tailwind setup
- component generation
- theme export
- static page templates
- blog template support
- asset compilation
- WordPress installable ZIP

Explicitly Excluded From MVP

- Elementor compatibility
- visual drag builder
- AI generation
- WooCommerce

- advanced WP APIs
 - SSR
 - Next.js runtime
 - multi-theme support
-

11. Suggested Folder Structure

```
my-theme/
├── src/
│   ├── components/
│   ├── pages/
│   ├── layouts/
│   ├── styles/
│   └── lib/
├── wordpress/
│   ├── templates/
│   ├── functions/
│   └── theme.json
├── dist/
├── wp.config.ts
└── package.json
```

12. Development Phases

Phase 1 — Foundation

Goal

Set up the base framework architecture.

Tasks

- Initialize monorepo

- Configure Vite
- Configure React + TS
- Configure Tailwind
- Create basic CLI
- Create folder structure
- Create build pipeline

Deliverables

- `create-wp-stack`
 - development server
 - build command
 - starter project
-

Phase 2 — Theme Compiler

Goal

Generate WordPress-compatible theme output.

Tasks

- Create template generator
- Generate:
 - `header.php`
 - `footer.php`
 - `index.php`
 - `functions.php`
 - `theme.json`
- Asset manifest generation
- ZIP export

Deliverables

- installable WordPress theme
 - working theme compilation
-

Phase 3 — Component System

Goal

Create reusable component workflow.

Tasks

- Build component registry
- Create CLI add command
- Create starter UI components
- Create layout templates
- Create theme presets

Deliverables

- `wp-add component`
 - reusable UI ecosystem
-

Phase 4 — WordPress Data Layer

Goal

Connect React components to WP data.

Tasks

- WP hooks abstraction
- WP Query abstraction
- blog loop utilities
- menu utilities
- custom field mapping

Deliverables

- dynamic content support
-

Phase 5 — Gutenberg Integration

Goal

Support native WordPress editing.

Tasks

- block registration
- block rendering
- editor compatibility
- dynamic block support

Deliverables

- Gutenberg-ready themes
-

Phase 6 — Ecosystem Expansion

Goal

Expand framework capabilities.

Future Ideas

- WooCommerce support
 - visual builder
 - AI generation
 - cloud deployment
 - component marketplace
 - theme marketplace
-

13. Suggested CLI Commands

Project Initialization

`npx create-wp-stack my-theme`

Development

`pnpm dev`

Build Theme

`pnpm build`

Export WordPress Theme

`pnpm export`

Add Components

`pnpm wp:add hero-section`

`pnpm wp:add navbar`

`pnpm wp:add pricing-table`

14. Internal Engineering Goals

Performance Goals

- lightweight output
 - minimal hydration
 - optimized assets
 - SEO-friendly rendering
-

DX Goals

- fast setup
 - clean architecture
 - modern tooling
 - simple commands
 - low PHP exposure
-

Compatibility Goals

- standard WordPress hosting
 - shared hosting support
 - plugin compatibility
 - native WP install flow
-

15. Major Risks & Challenges

Technical Risks

- WP dynamic data mapping
 - Gutenberg complexity
 - CSS conflicts
 - plugin compatibility
 - PHP template generation
-

Product Risks

- overengineering
 - trying to compete with Elementor too early
 - excessive abstraction
 - bloated runtime
-

16. Recommended Initial Priorities

First 30 Days

Focus ONLY on:

- framework setup
 - compiler architecture
 - basic theme export
-

Do NOT Build Yet

- visual builder
 - AI features
 - Elementor support
 - marketplace
 - advanced integrations
-

17. Long-Term Vision

The long-term goal is to become:

the modern frontend standard for WordPress theme engineering.

Not replacing WordPress —

but modernizing how developers build on top of it.

18. Recommended Immediate Next Steps

Step 1

Create:

- GitHub organization
 - monorepo
 - project naming
-

Step 2

Build:

- CLI scaffolder
 - Vite starter
 - React/Tailwind setup
-

Step 3

Create:

- first theme compiler prototype

Goal:

Generate a basic installable WP theme from React source files.

19. Success Criteria For MVP

The MVP is successful if:

- a React developer can build a WordPress theme
 - the theme installs normally
 - Tailwind works properly
 - reusable components exist
 - no advanced PHP knowledge is required
-

20. Final Product Positioning

Positioning Statement

A modern React-powered frontend framework for building native WordPress themes using Tailwind, reusable components, and modern developer tooling.

21. Core Identity Reminder

This product is NOT:

- a page builder
- a no-code platform
- a replacement for WordPress

This product IS:

- a developer framework
- a theme compiler
- a modern frontend workflow for WordPress developers and React engineers.